

04_features_repo

July 8, 2026

4. Image Features on CIFAR-10

This notebook adapts the original CS231n image-features exercise to the **local repo-friendly workflow** used by the other notebooks in **demos/**.

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of the work is still done in this notebook, but the code path is adapted for this repo: - work locally from a notebook in **demos/** - reuse implementations from **src/** - keep the original feature-engineering workflow intact - compare a linear model and a shallow neural network on the same feature representation

4.1 Related source files

- `src/features.py`
- `src/models/linear_classifier.py`
- `src/models/fc_net.py`
- `src/utils/data.py`
- `src/utils/solver.py`

4.2 Why use hand-crafted image features?

Raw pixels are easy to feed into a classifier, but they are not always the most informative representation. In this notebook we combine two classic feature families:

- **HOG (Histogram of Oriented Gradients)** to emphasize local edge and texture structure,
- **HSV hue histograms** to summarize coarse color information.

The point is not that these features beat modern deep vision models. The point is that they make the representation-design question concrete: **better inputs can make simple models much stronger.**

4.3 Setup

The setup below mirrors the other repo notebooks: - add the repository root to `sys.path`, - import local code from `src/`, - create output directories used by the model save helpers, - and configure plotting for notebook work.

```
[ ]: import sys
      from pathlib import Path

      repo_root = Path().resolve().parent
      if str(repo_root) not in sys.path:
          sys.path.append(str(repo_root))

      import random
      import tarfile
      import urllib.request

      import matplotlib.pyplot as plt
      import numpy as np

      from src.features import color_histogram_hsv, extract_features, hog_feature
      from src.models.fc_net import TwoLayerNet
      from src.models.linear_classifier import Softmax
      from src.utils.data import load_CIFAR10
      from src.utils.solver import Solver

      random.seed(231)
      np.random.seed(231)

      (repo_root / 'artifacts').mkdir(parents=True, exist_ok=True)

      %matplotlib inline
      plt.rcParams['figure.figsize'] = (10.0, 8.0)
      plt.rcParams['image.interpolation'] = 'nearest'
      plt.rcParams['image.cmap'] = 'gray'

      %load_ext autoreload
      %autoreload 2
```

4.4 Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

The only repo-specific difference is that we load it from the local `data/` directory instead of the original course layout, and keep the images in (N, 32, 32, 3) format since the feature extractors operate on standard HWC image arrays.

```
[ ]: cifar10_dir = repo_root / 'data' / 'cifar-10-batches-py'
```

```

if not cifar10_dir.exists():
    print('CIFAR-10 dataset not found. Downloading...')

    data_dir = repo_root / 'data'
    data_dir.mkdir(parents=True, exist_ok=True)

    url = 'https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz'
    archive_path = data_dir / 'cifar-10-python.tar.gz'

    urllib.request.urlretrieve(url, archive_path)

    print('Extracting dataset...')
    with tarfile.open(archive_path, 'r:gz') as tar:
        tar.extractall(path=data_dir)

    print('Download complete.')

for name in ['X_train', 'y_train', 'X_test', 'y_test']:
    if name in globals():
        del globals()[name]

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

print('Training data shape: ', X_train.shape)
print('Training labels shape:', y_train.shape)
print('Test data shape:      ', X_test.shape)
print('Test labels shape:   ', y_test.shape)

```

4.5 Split the data

We follow the same train / validation / test split used in the original assignment: - 49,000 training images, - 1,000 validation images, - 1,000 test images.

The validation split is what we will use for hyperparameter search.

```

[ ]: num_training = 49000
    num_validation = 1000
    num_test = 1000

    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]

    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]

    mask = list(range(num_test))

```

```

X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape:      ', X_train.shape)
print('Train labels shape:    ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels:     ', y_val.shape)
print('Test data shape:       ', X_test.shape)
print('Test labels shape:     ', y_test.shape)

```

4.6 Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The `extract_features` function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each row is the concatenation of all feature vectors for a single image.

In this repo version those helpers live in `src/features.py`, but the modeling idea is unchanged from the original notebook.

```

[ ]: num_color_bins = 25
    feature_fns = [
        hog_feature,
        lambda img: color_histogram_hsv(img, nbins=num_color_bins),
    ]

X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

std_feat = np.std(X_train_feats, axis=0, keepdims=True)
std_feat[std_feat < 1e-8] = 1.0
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

```

```

X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])

print('Feature matrix shapes:')
print('X_train_feats:', X_train_feats.shape)
print('X_val_feats: ', X_val_feats.shape)
print('X_test_feats: ', X_test_feats.shape)

```

Why normalize the feature coordinates?

The HOG block and the color-histogram block naturally live on different scales. Without normalization, optimization can be dominated by whichever coordinates happen to have the largest magnitudes. Standardizing the features makes the learning-rate and regularization search much more stable.

4.7 Train Softmax classifier on features

Using the Softmax code developed earlier in the assignment, train Softmax classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

The important point here is that the model family is still linear in the feature vector. The expected improvement comes from the representation, not from making the classifier itself more expressive.

```

[ ]: learning_rates = [1e-7, 5e-7, 1e-6]
    regularization_strengths = [5e5, 1e6, 5e6]

    results = {}
    best_val = -1.0
    best_softmax = None
    best_softmax_stats = None

    for lr in learning_rates:
        for reg in regularization_strengths:
            model = Softmax()
            model.train(
                X_train_feats,
                y_train,
                learning_rate=lr,
                reg=reg,
                num_iters=2000,
                batch_size=200,
                verbose=False,
            )

            train_pred = model.predict(X_train_feats)

```

```

val_pred = model.predict(X_val_feats)

train_acc = np.mean(y_train == train_pred)
val_acc = np.mean(y_val == val_pred)
results[(lr, reg)] = (train_acc, val_acc)

if val_acc > best_val:
    best_val = val_acc
    best_softmax = model
    best_softmax_stats = {
        'learning_rate': lr,
        'reg': reg,
        'train_acc': train_acc,
        'val_acc': val_acc,
    }

for (lr, reg), (train_acc, val_acc) in sorted(results.items()):
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (lr, reg,
    ↪ train_acc, val_acc))

print('\nbest validation accuracy:', best_val)
print('best_softmax_stats:', best_softmax_stats)

```

4.8 Final Softmax evaluation

A reasonably tuned model should comfortably outperform the raw-pixel linear baselines and typically land around or above **42% test accuracy**.

```

[ ]: y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax test accuracy:', test_accuracy)

```

```

[ ]: # Save best softmax model
best_softmax.save('best_softmax_features.npy')

```

4.9 Visualize misclassifications

Looking at mistakes is often more informative than looking at a single scalar accuracy. The visualization below groups false positives by predicted class so that we can see which visual confusions recur.

```

[ ]: examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
    ↪ 'ship', 'truck']

for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    if len(idxs) == 0:

```

```

        continue
    sample_size = min(examples_per_class, len(idxs))
    idxs = np.random.choice(idxs, sample_size, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls + 1)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)

plt.show()

```

Inline question 1:

Describe the misclassification results that you see. Do they make sense?

Your Answer:

The misclassifications often involve classes with similar global features. For example, trucks and cars are frequently confused because they share similar HOG edge structure and sometimes similar color distributions. Birds may also be misclassified as planes when the background is blue or when the silhouette is elongated in a way that HOG treats similarly. These mistakes make sense because the hand-crafted features focus on texture and coarse color information, but do not capture higher-level semantic parts as effectively as deeper learned representations.

Discussion: what mistakes should we expect?

The most common confusions usually involve classes that overlap in either: - **global shape** as seen by HOG, or - **dominant color distribution** as seen by the hue histogram.

So it is reasonable to see confusions like: - cars vs trucks, - birds vs planes against blue backgrounds, - deer vs horses in natural scenes.

Those errors make sense because these hand-crafted features capture edges, texture, and coarse color, but they do not encode higher-level semantic parts as effectively as deeper learned representations.

4.10 Train a two-layer network on the same features

Earlier in the assignment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; strong runs can get close to 60%.

In the repo version we remove the bias column added for the linear classifier and then train a `TwoLayerNet` on the standardized feature vectors themselves.

```
[ ]: X_train_feats_nn = X_train_feats[:, :-1].copy()
X_val_feats_nn = X_val_feats[:, :-1].copy()
X_test_feats_nn = X_test_feats[:, :-1].copy()

print('Neural-net feature shapes:')
print('X_train_feats_nn:', X_train_feats_nn.shape)
print('X_val_feats_nn: ', X_val_feats_nn.shape)
print('X_test_feats_nn: ', X_test_feats_nn.shape)
```

```
[ ]: input_dim = X_train_feats_nn.shape[1]
num_classes = 10

data = {
    'X_train': X_train_feats_nn,
    'y_train': y_train,
    'X_val': X_val_feats_nn,
    'y_val': y_val,
    'X_test': X_test_feats_nn,
    'y_test': y_test,
}

hidden_dims = [500, 1000]
learning_rates = [0.5, 1.0]
regularization_strengths = [1e-4, 1e-3]

best_net = None
best_net_stats = None
best_net_val = -1.0

for hidden_dim in hidden_dims:
    for lr in learning_rates:
        for reg in regularization_strengths:
            net = TwoLayerNet(
                input_dim=input_dim,
                hidden_dim=hidden_dim,
                num_classes=num_classes,
                weight_scale=1e-3,
                reg=reg,
            )
            solver = Solver(
                net,
                data,
                update_rule='sgd',
                optim_config={'learning_rate': lr},
                lr_decay=0.95,
                num_epochs=10,
                batch_size=200,
```



```

        print_every=100,
        verbose=False,
    )
    solver.train()

    train_acc = solver.train_acc_history[-1]
    val_acc = solver.best_val_acc

    print(
        f'hidden_dim={hidden_dim}, lr={lr}, reg={reg}, '
        f'train_acc={train_acc:.4f}, val_acc={val_acc:.4f}'
    )

    if val_acc > best_net_val:
        best_net_val = val_acc
        best_net = net
        best_net_stats = {
            'hidden_dim': hidden_dim,
            'learning_rate': lr,
            'reg': reg,
            'train_acc': train_acc,
            'val_acc': val_acc,
        }

print('\nbest two-layer validation accuracy:', best_net_val)
print('best_net_stats:', best_net_stats)

```

4.11 Final neural-network evaluation

A tuned two-layer network on top of these features should usually beat the linear feature classifier and often reach **58%+ test accuracy**.

```

[ ]: y_val_pred = np.argmax(best_net.loss(data['X_val']), axis=1)
     y_test_pred_nn = np.argmax(best_net.loss(data['X_test']), axis=1)

     print('Two-layer validation accuracy:', np.mean(y_val_pred == data['y_val']))
     print('Two-layer test accuracy:      ', np.mean(y_test_pred_nn ==
     ↪data['y_test']))

```

```

[ ]: # Save best two-layer model
     best_net.save('best_two_layer_net_features.npy')

```

4.12 Takeaways

We have now compared two different model families on the same hand-crafted feature representation.

- A better representation can dramatically improve a simple classifier.

- HOG and hue histograms capture useful low-level structure that raw pixels hide.
- A shallow nonlinear network on top of engineered features can improve performance further.
- Inspecting mistakes is essential: accuracy tells us how much error remains, while qualitative plots help explain why.

This is one of the main lessons of the notebook: even before using deeper models, the choice of representation can change the quality of the classifier substantially.